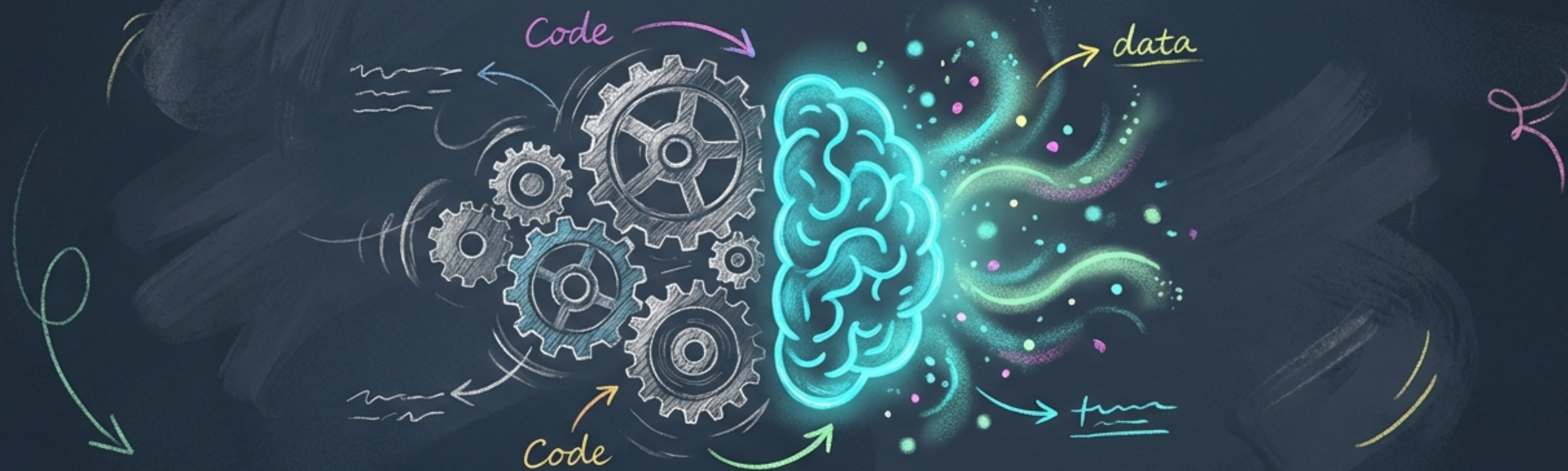


借助大模型API自动处理任务



一份写给程序员的AI赋能实战指南

我们的探索之旅



1. 面临的挑战：
当海量非结构化
数据涌来



2. 神兵利器：选择
模型并获取API密钥



3. 施展魔法：
编写Python代码，
实现自动化

3. 施展魔法：
编写Python代码，
实现自动化



4. 新的视野：总
结与更多应用场景

程序员的困境：当产品调研遇到一万份主观反馈

背景 (Background)

收到上万份产品调研问卷的主观反馈。

任务 (Task)

统计用户认为需要改进的方面，例如：价格过高、售后支持不足、产品使用体验不佳等。

痛点 (Pain Point)

- ☆ **现状：** 超过10000份主观反馈，内容未经结构化。
- ◎ **难点：** 要求在1天内做出分析报告。人工处理？几乎不可能。



新的编程范式：用API调用“AI大脑”

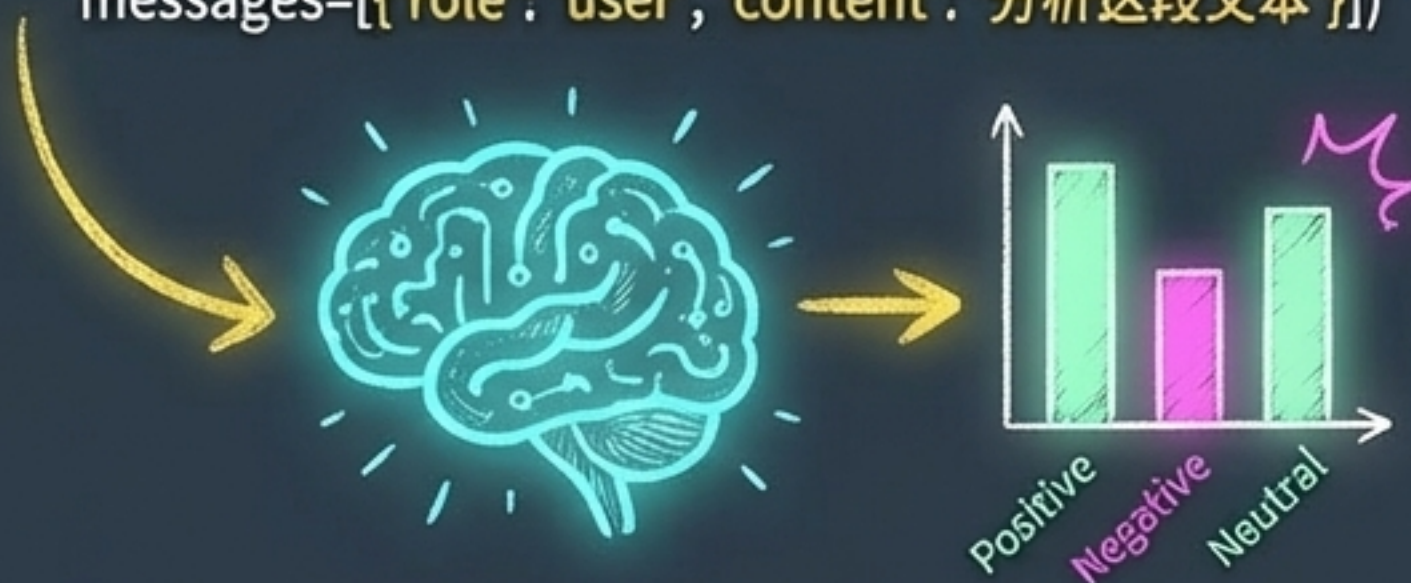
传统方法



人工阅读、归类，效率低下。

大模型API方法

```
response = client.chat.completions.create(model='gpt-4',  
messages=[{'role': 'user', 'content': '分析这段文本'}])
```



单行代码调用，即时输出结构化洞察。

核心理念 (Core Idea)：借助大语言模型API，可以极大地简化和加速文本处理的任务。

转变 (The Shift)：从逐条阅读、手动归类，到编写脚本、批量调用。这不仅仅是提效，更是工作方式的变革。

选择你的AI同盟：主流大模型概览



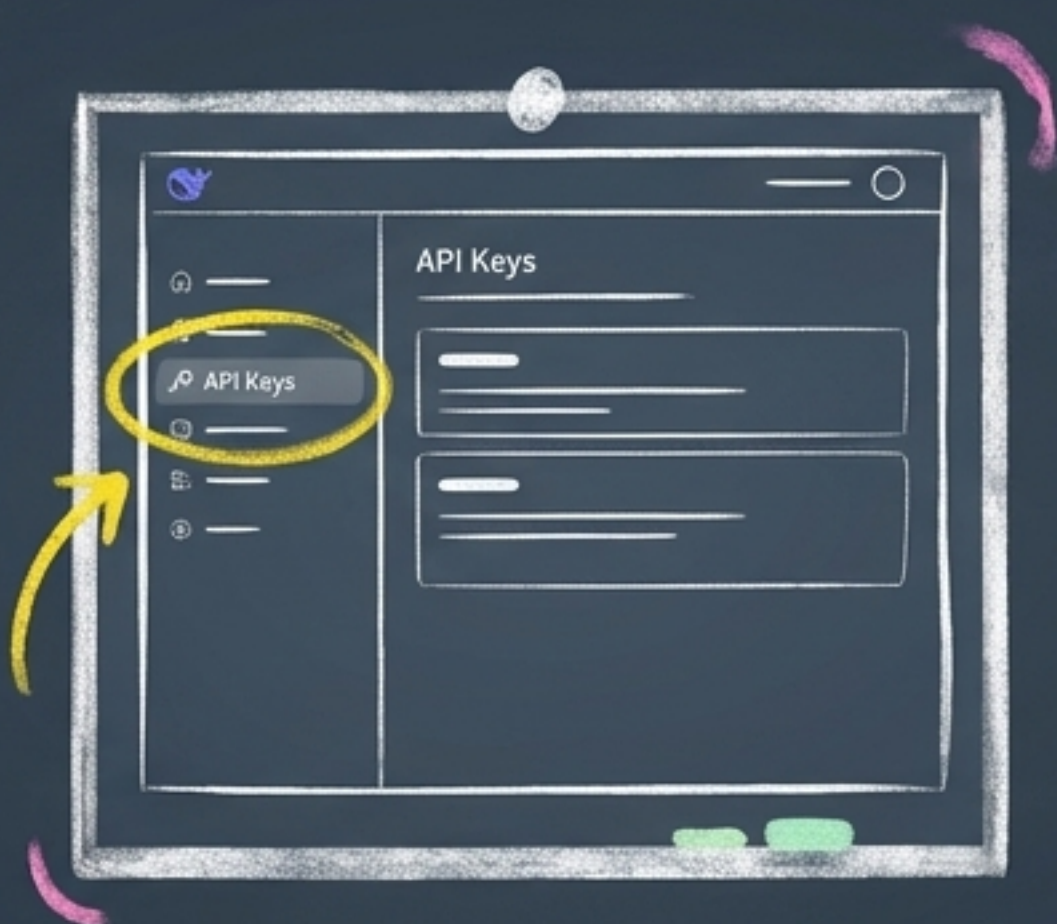
当前有许多优秀的大模型可供选择，它们大多提供API服务。

本次演示，我们将以 DeepSeek 模型 为例。它的API与OpenAI格式兼容，便于我们快速上手。

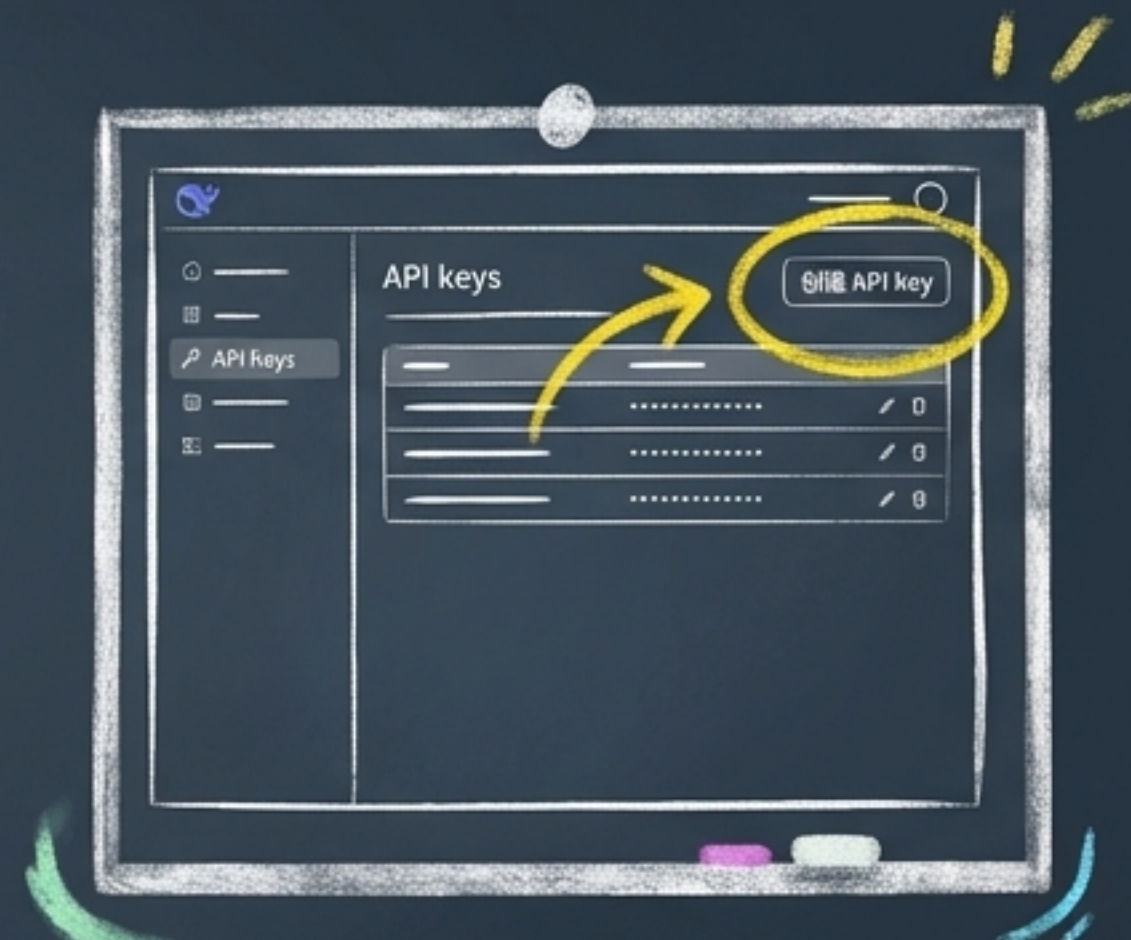
探险第一步：获取你的专属API密钥



第一步：访问官网，找到
API平台入口

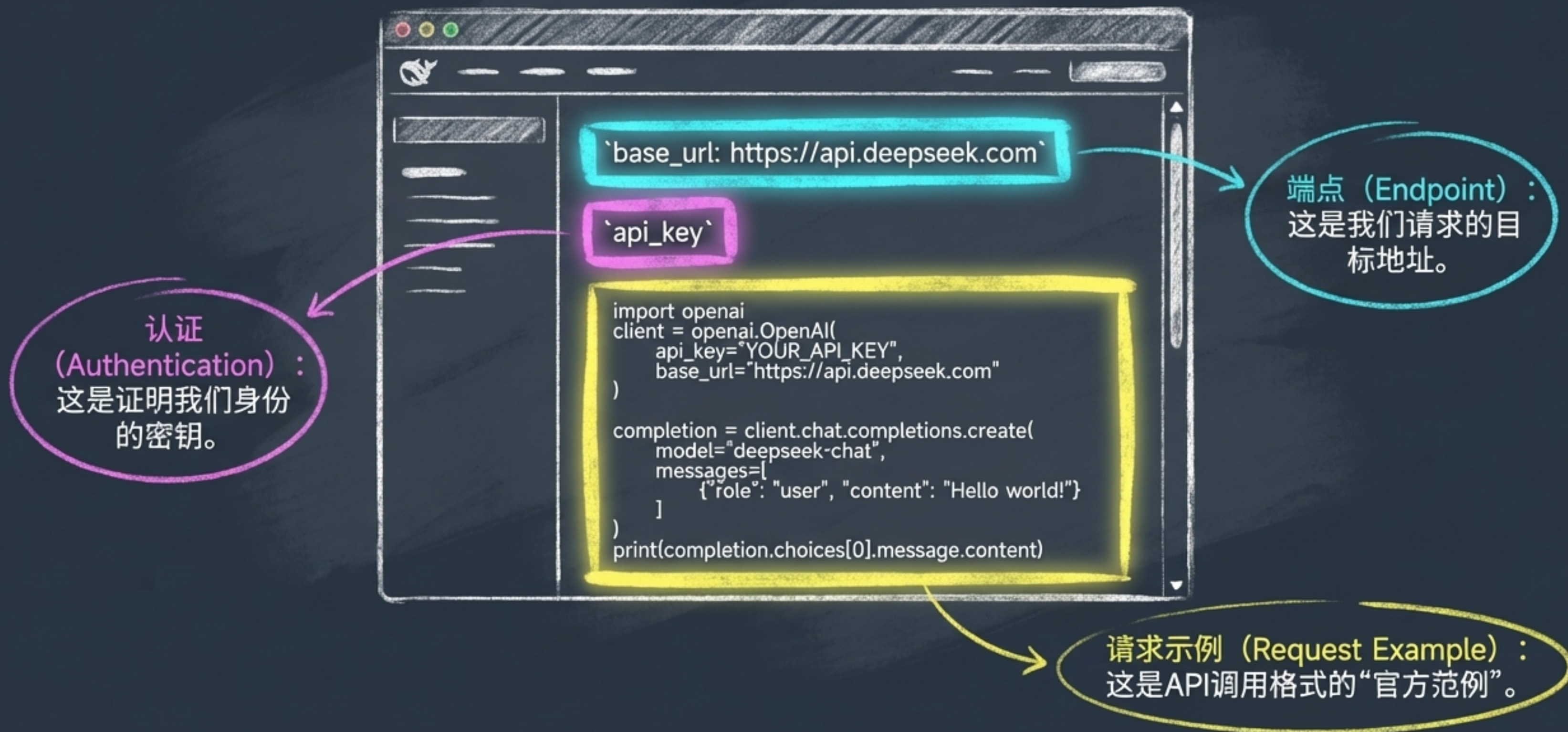


第二步：进入管理后台

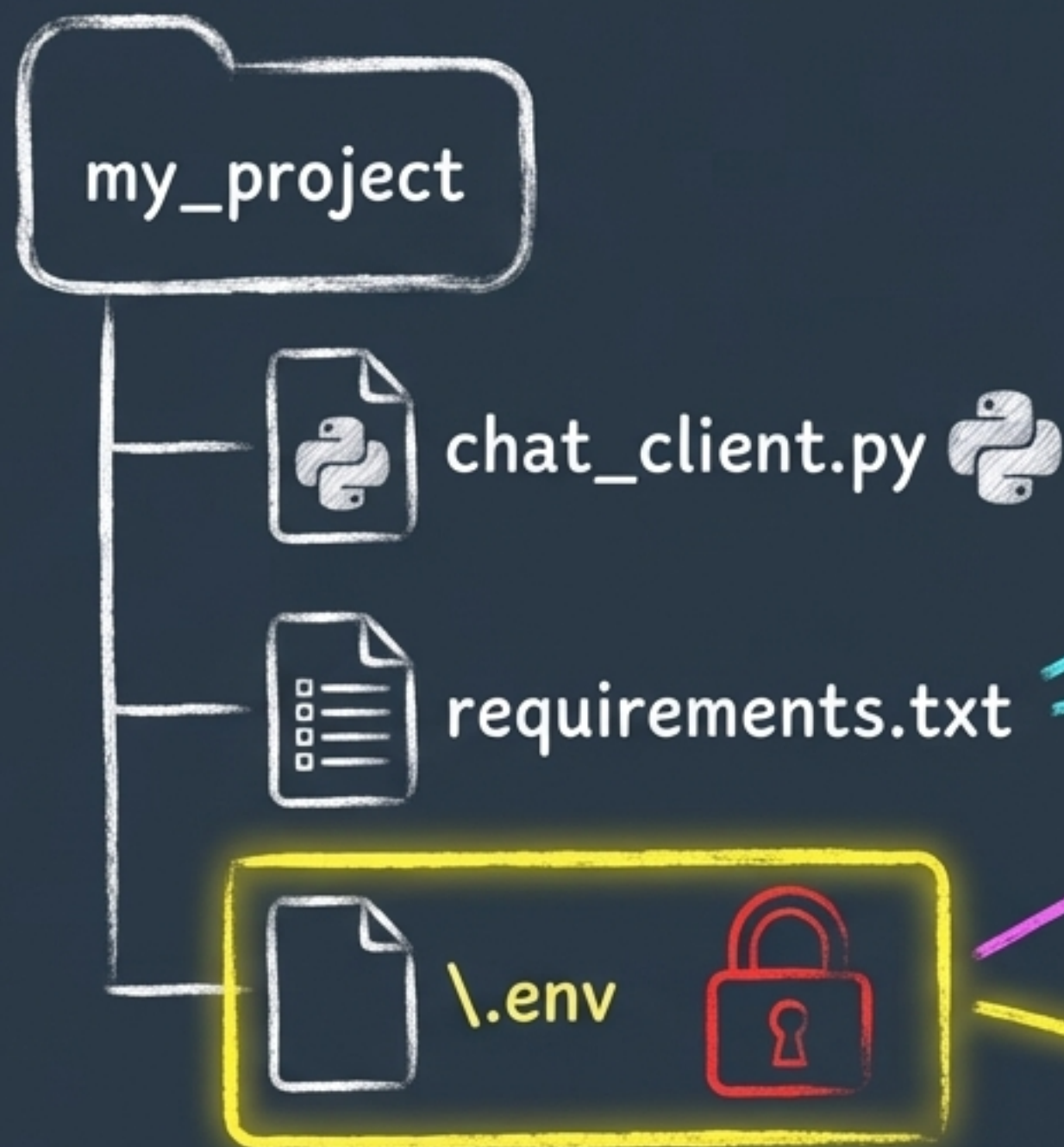


第三步：创建并妥善保管
你的密钥

解读藏宝图：API文档的核心三要素



搭建工作环境：专业且安全



项目结构 (Project Structure) :

- **chat_client.py**: 我们的主逻辑脚本。
- **requirements.txt**: 管理项目依赖。
 - **openai** (许多模型兼容其SDK)
 - **python-dotenv** (用于安全加载密钥)
- **.env**: **[重点]** 存储敏感信息, **绝不提交到代码库!**

```
DEEPSEEK_API_KEY="sk-..."  
DEEPSEEK_BASE_URL="https://api.deepseek.com"
```


施展魔法(一): 导入工具库与加载密钥

导入所需工具库

import os ①

from dotenv import load_dotenv ②

from openai import OpenAI ③

执行加载动作, 让.env文件中的变量生效

load_dotenv() ④

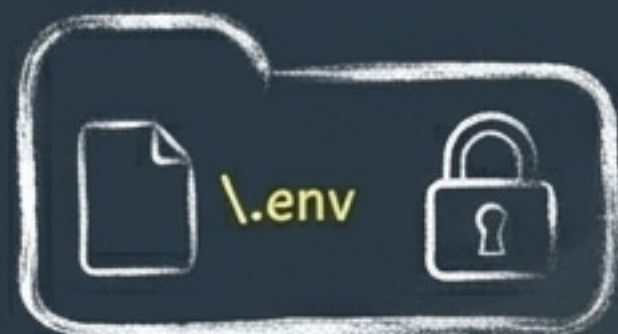
读取系统中的变量。

魔法的起点, 让`.env`文件“活”起来。

我们与大模型沟通的“翻译器”。

运行! 现在我们可以安全地使用密钥了。

施展魔法(二): 建立与大模型的连接



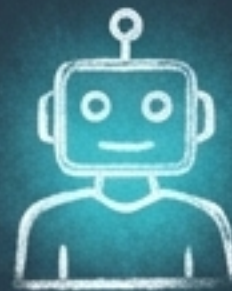
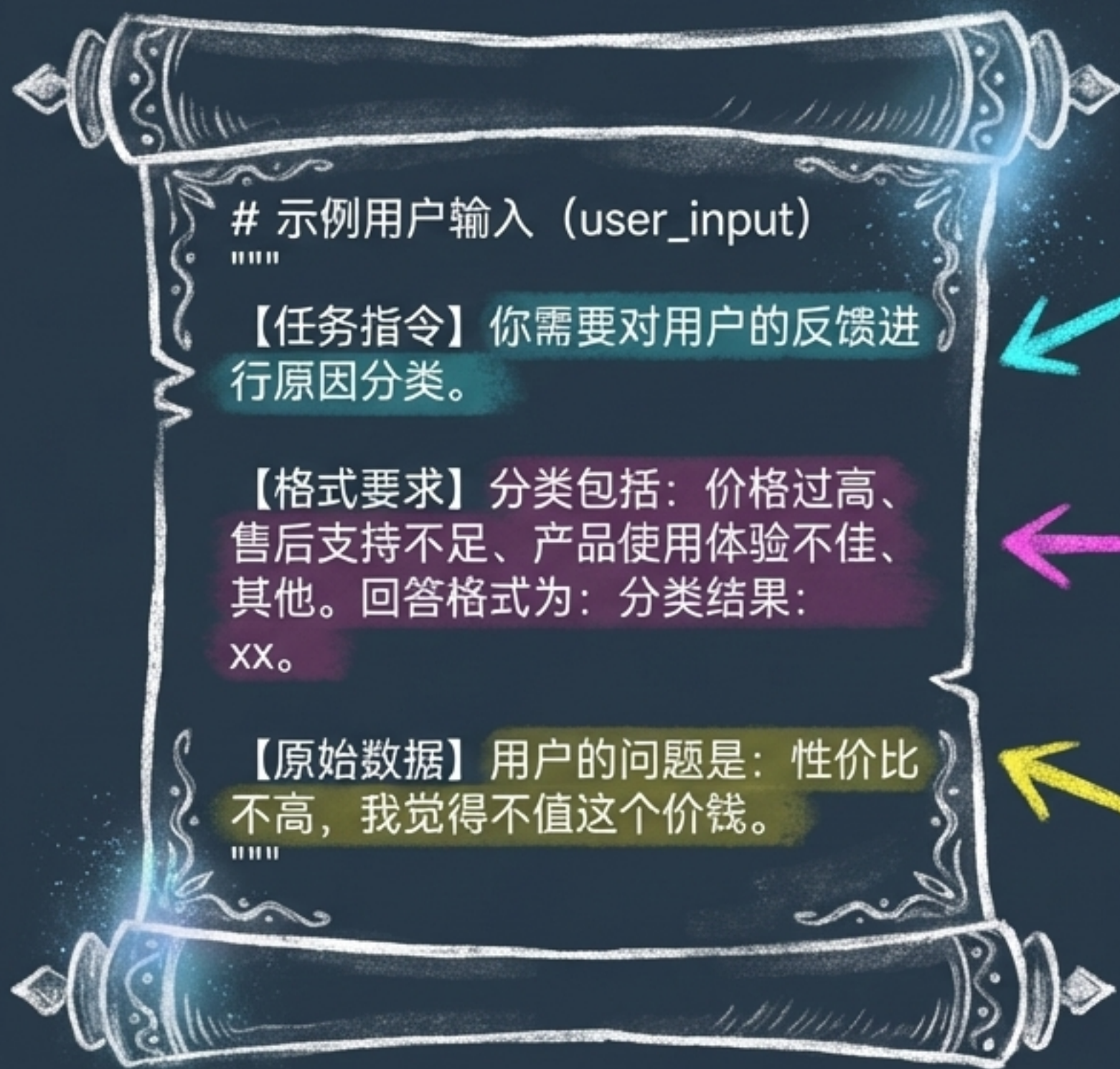
```
# 导入所需工具库
import os
from dotenv import load_dotenv
from openai import OpenAI
# 执行加载动作
load_dotenv()

# 初始化客户端
# 程序会从环境中安全地读取密钥和URL
client = OpenAI(
    api_key=os.getenv('DEEPSEEK_API_KEY'),
    base_url=os.getenv('DEEPSEEK_BASE_URL')
)
```

`os.getenv('DEEPSEEK\_API\_KEY')` 会自动、安全地从你的环境中（由`load\_dotenv()`加载）取出API密钥。

最佳实践：这种方式可以避免将你的密钥硬编码在代码中，防止意外泄露。

咒语的核心：精心设计你的指令（Prompt）



清晰的角色与任务 (Clear Role & Task)：告诉AI它是什么，要做什么。



明确的约束与格式 (Specific Constraints & Format)：定义你想要的输出，减少不确定性。



分隔的上下文与数据 (Separated Context & Data)：将指令和待处理数据清晰分开。

施展魔法(三): 发送请求, 获取智慧

① 告诉API使用哪个“大脑”。

② 模拟一次用户对话。

③ 这里传入了上一
页的完整指令。

```
# (Previous code is visible but ..)
response = client.chat.completions()

# 创建聊天完成请求
response = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {
            "role": "user",
            "content": user_input
        }
    ]
)

# 打印出模型返回的结果
print(response.choices[0].message.content)
```

④

从复杂的响应数据中
提取出我们最关心的
文本内容。

见证奇迹：从杂乱文本到结构化洞察

Before ➡ After

****输入 (Input) ****

你需要对用户的反馈进行原因分类。
分类包括：价格过高、售后支持不足、产品使用体验不佳、其他。
回答格式为：分类结果：xx。
用户的问题是：性价比不高，我觉得不值这个价钱。



****输出 (Output) **** _ □ ×

```
$ python chat_client.py  
分类结果：价格过高
```

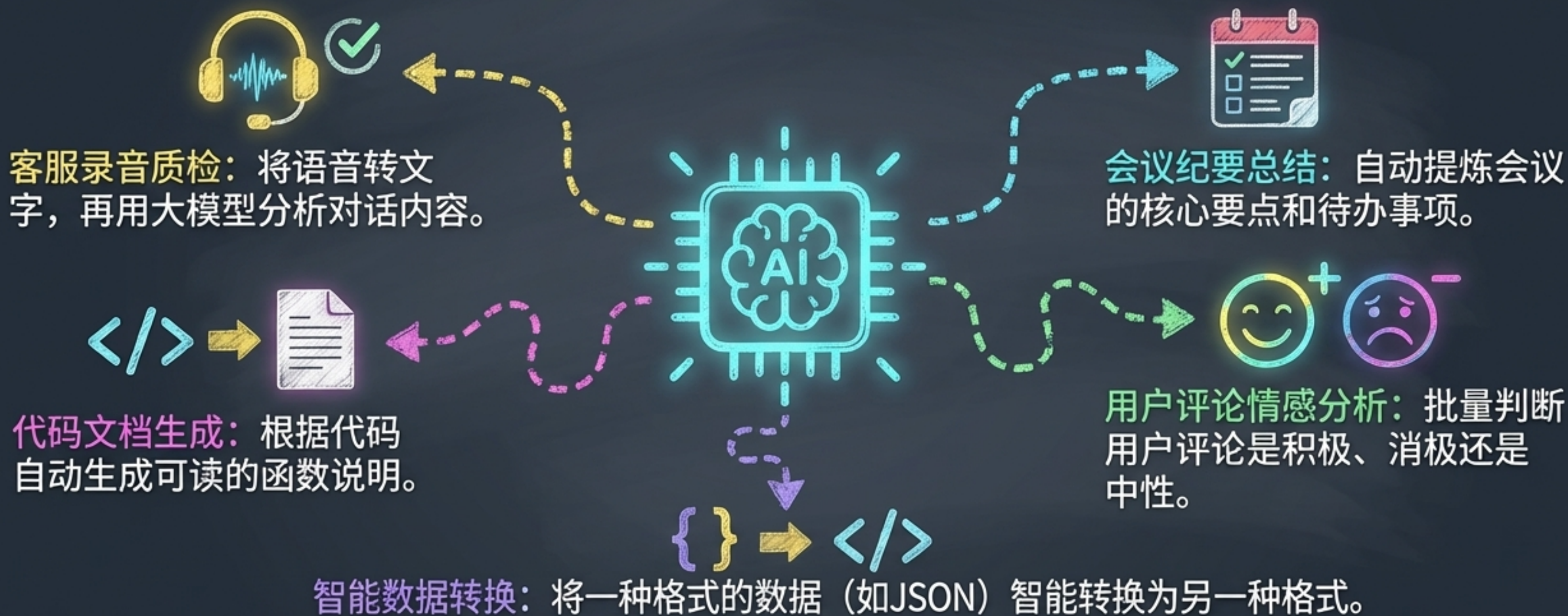
成功！我们用几行代码，瞬间完成了需要人工数分钟甚至更久才能完成的分析与归类工作。

任务完成：我们刚刚实现了什么？



- ✓ 我们面对了海量非结构化文本的挑战。
- ✓ 我们选择了一个大模型，并通过API文档掌握了它的使用方法。
- ✓ 我们编写了不到30行的Python代码，并遵循了安全编码的最佳实践。
- ✓ 最终，我们实现了对任意文本的自动分类，将“混乱”转化为了“洞察”。

你的下一场冒险：大模型API还能做什么？



你已经掌握了钥匙。现在，去开启属于你的自动化大门吧。